PHASE 7 OF 18

OOP II: Polymorphism, Abstraction & Interfaces

Overloading and its resolution rules, overriding rules, covariant returns, dynamic binding, method hiding, reference casting, final, abstract classes, interfaces (default/static/private), marker and functional interfaces

IN THIS PHASE

7.1 Polymorphism
7.2 Method Overloading
7.3 Method Overriding
7.4 Dynamic Binding (Runtime Polymorphism)
7.5 Reference Casting: Upcasting and Downcasting
7.6 The final Keyword
7.7 Abstraction and Abstract Classes
7.8 Interfaces
7.9 Abstract Class vs Interface
7.10 Marker and Functional Interfaces
7.11 Predict the Output
7.12 Interview Questions
7.13 Quick Revision

7.1 Polymorphism

Polymorphism ("many forms") means one name or one reference can behave differently depending on the situation. Even operators show it: `10 + 20` adds, `"Hi" + "Hello"` concatenates.

	Compile-time (static) polymorphism	Runtime (dynamic) polymorphism
Achieved by	Method **overloading** (and constructor overloading)	Method **overriding**
Decided by	The **compiler**, from argument types	The **JVM**, from the actual object type
Binding	Static (early) binding	Dynamic (late) binding
Needs inheritance?	No (same class is enough)	Yes (parent–child or interface–class)
Example	`add(int, int)` vs `add(double, double)`	`Payment p = new UpiPayment(); p.pay();`

7.2 Method Overloading

Overloading means several methods in the same class (or inherited into it) share a **name** but have **different parameter lists**.

```
class Calculator {
    int add(int a, int b)             { return a + b; }
    int add(int a, int b, int c)      { return a + b + c; }   // different count
    double add(double a, double b)    { return a + b; }       // different types
    String add(String a, int b)       { return a + b; }       // different order...
    String add(int b, String a)       { return b + a; }       // ...also counts
    // long add(int a, int b) { }     // COMPILE ERROR: only the return type differs
}
```

Rules

1. Parameter lists **must differ** in number, type or order of types.

2. Return type, access modifier, `static` and the `throws` clause **may** differ but don't count on their own.
3. Overloading works for static methods, constructors and `main` (the JVM only calls `main(String[])`).
4. The compiler decides which overload to call **at compile time**, based on the **declared (static) types** of the arguments.

How the compiler picks an overload

When there is no exact match, the compiler tries each phase in order and stops at the first phase that finds a candidate:

Phase	Allows	Example: call `m(5)` with an `int`
1. Exact match	Identical types	`m(int)`
2. Widening	Primitive widening, subtype → supertype	`m(long)`, `m(double)`
3. Boxing / unboxing	`int` ↔ `Integer`	`m(Integer)`, `m(Object)`
4. Varargs	`int...`	`m(int...)`

Within a phase, the **most specific** candidate wins (for example `m(long)` beats `m(double)`, and `m(String)` beats `m(Object)`).

```
static void m(long x)    { System.out.println("long"); }
static void m(Integer x) { System.out.println("Integer"); }
static void m(int... x)  { System.out.println("varargs"); }
m(5);                    // long    (widening beats boxing)

static void t(Object o)  { System.out.println("Object"); }
static void t(String s)  { System.out.println("String"); }
t(null);                 // String  (most specific type)
t("hi");                 // String
Object o = "hi";
t(o);                    // Object  (decided by the DECLARED type, not the runtime type!)

static void u(String s)        { }
static void u(StringBuilder s) { }
// u(null);              // COMPILE ERROR: ambiguous (neither type is more specific)
```

PITFALL

`List<Integer> list` has both `remove(int index)` and `remove(Object o)`. `list.remove(1)` removes the element **at index 1**. To remove the *value* 1, call `list.remove(Integer.valueOf(1))`.

7.3 Method Overriding

Overriding means a subclass provides its **own implementation** of an inherited instance method with the **same signature**. At runtime, the JVM calls the version belonging to the **actual object**.

```
class Account {
    double interestRate() { return 0.0; }
    String type() { return "Generic"; }
}
class SavingsAccount extends Account {
    @Override double interestRate() { return 4.0; }
    @Override String type() { return "Savings"; }
}
class FixedDeposit extends Account {
    @Override double interestRate() { return 7.1; }
}

Account a = new SavingsAccount();
System.out.println(a.type() + " " + a.interestRate());   // Savings 4.0
```

Overriding rules

Rule	Detail
Same name and same parameter list	Otherwise it is overloading, not overriding
Return type	Same, or a **subtype** (covariant return) for objects. Primitives must match exactly.
Access modifier	**Same or wider**, never narrower (`protected` → `public` is OK; `public` → `protected` is an error)
Checked exceptions	May throw **fewer, narrower or no** checked exceptions, never new or broader ones. Unchecked exceptions are unrestricted.
`final` methods	Cannot be overridden
`static` methods	Cannot be overridden; a same-signature static method in the child **hides** the parent's
`private` methods	Not inherited, so not overridden (a same-name child method is a new method)
Constructors	Not inherited, so never overridden
Instance relationship	Requires inheritance (`extends` or `implements`)

```
class Parent {
    protected Number value() throws java.io.IOException { return 1; }
}
class Child extends Parent {
    @Override
    public Integer value() { return 2; }   // wider access, covariant, no checked exc.
}

class Bad extends Parent {
    // @Override private Number value() { }                        // ERROR: weaker access
    // @Override Number value() throws Exception { return 0; }     // ERROR: broader checked
}
```

> **TIP**
> Always use `@Override`. If the method doesn't actually override anything (typo in the name, wrong parameter type such as `equals(Student)` instead of `equals(Object)`), the compiler reports an error instead of silently creating a new method.

Covariant return types (Java 5)

An overriding method may return a **subtype** of the parent method's return type. This avoids casts for callers.

```
class Animal { Animal reproduce() { return new Animal(); } }
class Dog extends Animal {
    @Override Dog reproduce() { return new Dog(); }   // Dog is-a Animal
}
Dog puppy = new Dog().reproduce();                    // no cast needed
```

Overloading vs overriding

Feature	Overloading	Overriding
Purpose	Same operation, different inputs	Change inherited behaviour
Where	Same class (or inherited)	Parent ↔ child
Parameters	**Must differ**	**Must be the same**
Return type	Free to differ	Same or covariant
Access modifier	Free	Same or wider
Exceptions	Free	Not broader checked exceptions
`static` / `private` / `final`	Can be overloaded	Cannot be overridden
Resolved	Compile time (static binding)	Runtime (dynamic binding)
Polymorphism type	Compile-time	Runtime

7.4 Dynamic Binding (Runtime Polymorphism)

Binding links a method call to a method body.

- **Static (early) binding** happens at compile time. It applies to `private`, `static` and `final` methods, constructors, overloaded method selection, and **all field access**.
- **Dynamic (late) binding** happens at runtime. It applies to overridable **instance** methods: the JVM looks at the object's real class and calls its version (HotSpot uses a per-class method table, the *vtable*).

```
interface Payment { void pay(double amount); }

class UpiPayment implements Payment {
    public void pay(double amt) { System.out.println("UPI: " + amt); }
}
class CardPayment implements Payment {
    public void pay(double amt) { System.out.println("Card: " + amt); }
}
class NetBanking implements Payment {
    public void pay(double amt) { System.out.println("NetBanking: " + amt); }
}

Payment[] payments = { new UpiPayment(), new CardPayment(), new NetBanking() };
for (Payment p : payments) {
    p.pay(500);      // ONE call site, THREE behaviours -> chosen at runtime
}
```

> **NOTE**
> This is why polymorphism matters: code written against `Payment` keeps working when a new payment type (for example `WalletPayment`) is added, with **no changes** to existing callers. This is the *Open/Closed Principle*.

Method hiding (static methods)

```
class Parent { static void info() { System.out.println("Parent static"); } }
class Child extends Parent { static void info() { System.out.println("Child static"); } }

Parent p = new Child();
p.info();          // Parent static -> chosen by REFERENCE type (hiding, not overriding)
Child.info();      // Child static
```

Overriding (instance methods)	Hiding (static methods)
Chosen by **object** type at runtime	Chosen by **reference** type at compile time

Overriding (instance methods)	Hiding (static methods)
Polymorphic	Not polymorphic
`@Override` allowed	`@Override` not allowed (compile error)

7.5 Reference Casting: Upcasting and Downcasting

	Upcasting	Downcasting
Direction	Child → parent / interface type	Parent → child type
Syntax	Implicit: `Account a = new SavingsAccount();`	Explicit: `SavingsAccount s = (SavingsAccount) a;`
Safe?	Always	Only if the object really is that type; otherwise `ClassCastException` at runtime
Why	Generic code, polymorphism	Access child-only members

```
Account a = new SavingsAccount();        // upcast (automatic)
// a.addInterest();                      // COMPILE ERROR: not visible via Account

SavingsAccount s = (SavingsAccount) a;   // downcast: OK, the object IS a SavingsAccount
s.addInterest();

Account b = new FixedDeposit();
SavingsAccount bad = (SavingsAccount) b; // compiles, but ClassCastException at RUNTIME

// String str = (String) new Integer(5); // COMPILE ERROR: unrelated types can't be cast
```

Safe downcasting with instanceof

```
if (b instanceof SavingsAccount) {               // classic check, then cast
    SavingsAccount sv = (SavingsAccount) b;
    sv.addInterest();
}

if (b instanceof SavingsAccount sv) {            // Java 16 pattern matching: check + cast
    sv.addInterest();
}

String label = switch (b) {                      // Java 21 pattern matching for switch
    case SavingsAccount sv -> "Savings @" + sv.interestRate();
    case FixedDeposit fd   -> "FD @" + fd.interestRate();
    default                -> "Other";
};
```

REMEMBER

`obj instanceof Type` is `true` for that type **and all its supertypes**, and always `false` for `null`. `obj.getClass() == Type.class` is true only for the **exact** class.

TIP

Frequent `instanceof` + downcast chains are a design smell. Usually the behaviour belongs in an overridden method on the type itself.

7.6 The final Keyword

Applied to	Effect	Example
Local variable / parameter	Assigned once	`final int max = 10;`
Instance variable	Assigned once: at declaration, in an instance block, or in **every** constructor (a *blank final*)	`private final String accountNo;`
Static variable	Assigned once: at declaration or in a static block (a constant)	`static final double PI = 3.14159;`
Method	Cannot be overridden	`public final void audit() { }`
Class	Cannot be extended	`final class String`, `Integer`, `Math`, `LocalDate`

```
class Customer {
    private final String id;                  // blank final
    private static final int MAX_CARDS;       // static blank final
    static { MAX_CARDS = 3; }
    Customer(String id) { this.id = id; }     // must be assigned in every constructor
}

final List<String> names = new ArrayList<>();
names.add("Rohit");          // ALLOWED: the list object itself is mutable
// names = new ArrayList<>(); // ERROR: the REFERENCE can't be reassigned
```

CORRECTED FROM ORIGINAL NOTES

The original notes say "**final classes are immutable**". That's incorrect. `final` on a class only **prevents subclassing**. A final class can still have setters and mutable fields. Immutability requires `private final` fields, no mutators and defensive copies (Phase 8). `String` happens to be both final *and* immutable; being final is just one part of what makes it immutable.

NOTE

Effectively final: a local variable that is never reassigned, even without the `final` keyword. Lambdas and anonymous/local classes may only capture final or effectively final locals.

7.7 Abstraction and Abstract Classes

Abstraction means exposing **what** an object does while hiding **how** it does it. You use an ATM's Withdraw, Deposit and Balance operations without knowing the internal logic. In Java, abstraction is achieved with **abstract classes** (partial abstraction) and **interfaces** (full abstraction of behaviour).

- A **concrete method** has a body: `void display() { ... }`.
- An **abstract method** has no body and ends with `;`: `abstract double emi();`. It may appear only in an abstract class or an interface.

```
abstract class Loan {                                    // can't be instantiated
    protected final double principal;
    protected final int months;

    protected Loan(double principal, int months) {       // abstract classes HAVE constructors
        this.principal = principal;
        this.months = months;
    }

    abstract double annualRate();                        // subclasses must implement

    public final double emi() {                          // template method: fixed algorithm
        double r = annualRate() / 12 / 100;
        return principal * r * Math.pow(1 + r, months) / (Math.pow(1 + r, months) - 1);
    }
}

class HomeLoan extends Loan {
    HomeLoan(double p, int m) { super(p, m); }
    @Override double annualRate() { return 8.5; }
}
class CarLoan extends Loan {
    CarLoan(double p, int m) { super(p, m); }
    @Override double annualRate() { return 9.2; }
}

Loan loan = new HomeLoan(5_000_000, 240);                // Loan l = new Loan(...) won't compile
System.out.printf("EMI = %.2f%n", loan.emi());           // EMI = 43391.16
```

Abstract class rules

1. Declared with `abstract`; **cannot be instantiated** with `new` (anonymous subclasses are possible).
2. Can contain abstract **and** concrete methods, constructors, instance and static fields, static methods, blocks, even `main`.
3. Its constructor runs when a **subclass** object is created, through `super(...)`.
4. A concrete subclass must implement **all** inherited abstract methods; otherwise it must itself be declared `abstract`.
5. A class may be abstract with **zero** abstract methods (to prevent direct instantiation).
6. Illegal combinations: `abstract final` (class or method), `abstract private`, `abstract static` and `abstract synchronized` (methods).

7.8 Interfaces

An **interface** is a pure **contract**: it states *what* operations a type supports, not *how*. Classes **implement** interfaces; one class can implement **many** of them.

```
public interface Bank {
    double MIN_BALANCE = 1000;              // implicitly public static final
    void transfer(String to, double amt);   // implicitly public abstract
    double checkBalance();
}

class HdfcBank implements Bank {
    public void transfer(String to, double amt) { System.out.println("HDFC transfer"); }
    public double checkBalance() { return 5000; }
}
class AxisBank implements Bank {
    public void transfer(String to, double amt) { System.out.println("Axis transfer"); }
    public double checkBalance() { return 8000; }
}

Bank b = new HdfcBank();       // interface reference -> any implementation
b = new AxisBank();            // swap implementations freely
// Bank x = new Bank();        // COMPILE ERROR: interfaces can't be instantiated
```

Implicit modifiers

Member	What you write	What the compiler means
Field	`int LIMIT = 5;`	`public static final int LIMIT = 5;` (a constant; must be initialised)
Abstract method	`void pay();`	`public abstract void pay();`
Default method (Java 8)	`default void log() { }`	`public` instance method with a body
Static method (Java 8)	`static Bank of() { }`	`public static`, called **only** as `Bank.of()`
Private method (Java 9)	`private void helper() { }`	Shared helper for default/static methods
Constructor	not allowed	n/a
Instance fields	not allowed	n/a

> **REMEMBER**
> Implementing methods must be declared `public`, because interface methods are implicitly public and you can't reduce visibility.

How interfaces evolved

```
public interface PaymentGateway {
    void pay(double amount);                                  // abstract (Java 1.0+)

    default void payWithReceipt(double amount) {              // default (Java 8)
        pay(amount);
        log("Receipt generated for " + amount);
    }

    static PaymentGateway noop() {                            // static (Java 8)
        return amount -> { };                                 // lambda: one abstract method
    }

    private void log(String msg) {                            // private (Java 9)
        System.out.println("[gateway] " + msg);
    }
}
```

Default methods were added in Java 8 so that existing interfaces (such as `Collection`) could gain new methods (`stream()`, `forEach()`, `removeIf()`) **without breaking** every class that already implemented them.

Interface inheritance and the default-method diamond

```
interface RbiRules { void kyc(); }
interface Bank extends RbiRules { void openAccount(); }  // interfaces EXTEND interfaces
interface Insurance { }

class HdfcBank extends Object implements Bank, Insurance {   // extends first, then implements
    public void kyc() { }
    public void openAccount() { }
}

interface Camera { default String click() { return "Camera"; } }
interface Phone  { default String click() { return "Phone"; } }

class SmartPhone implements Camera, Phone {
    @Override
    public String click() {                      // MUST override (else compile error)
        return Camera.super.click() + "+" + Phone.super.click();   // InterfaceName.super.m()
    }
}
```

Relationship	Keyword	Multiple allowed?
class → class	`extends`	No (exactly one)
class → interface	`implements`	Yes
interface → interface	`extends`	Yes
interface → class	not possible	n/a

> **NOTE**
> Conflict resolution rules: (1) a method in a **class** (or superclass) always wins over interface defaults; (2) a more specific sub-interface wins over its parent; (3) otherwise the class must override and choose with `X.super.m()`.

Interfaces as types: loose coupling

```
class PaymentService {
    private final PaymentGateway gateway;         // depends on the CONTRACT
    PaymentService(PaymentGateway gateway) {      // implementation injected (e.g. by Spring)
        this.gateway = gateway;
    }
    void checkout(double amt) { gateway.pay(amt); }
}
```

```
Tight coupling:   PaymentService ----------> RazorpayGateway       (change = edit service)
Loose coupling:   PaymentService ---> <<PaymentGateway>> <--- RazorpayGateway
                                                          <--- StripeGateway
                                                          <--- MockGateway (unit tests!)
```

> **TIP**
> For a test engineer, this is the key benefit: code that depends on interfaces can receive **mocks or stubs** (with Mockito, for example) in unit tests, with no real payment gateway, database or network.

7.9 Abstract Class vs Interface

Feature	Abstract class	Interface
Instantiation	No	No
Methods	Abstract + concrete (any access)	Abstract, `default`, `static`, `private`
Fields	Any: instance, static, any access, mutable	Only `public static final` constants
Constructors	Yes	No
State per object	Yes	No
Inheritance	`extends` **one** class	A class `implements` **many**; an interface `extends` many
Access of methods	Any	Public (except private helpers)
Use for	**Is-a** base with shared state and code (a skeleton)	**Can-do** capability or contract (`Comparable`, `Runnable`)
Speed of adding methods	Add a concrete method safely	Add a `default` method safely

CORRECTED FROM ORIGINAL NOTES

The original notes say "from Java 8, there is no demand for abstract classes". Interfaces still **cannot hold instance state or constructors**, so abstract classes remain the right tool for a common base with shared fields and partially implemented logic (for example `AbstractList`, or `HttpServlet` in Jakarta EE). A common design is an **interface for the contract plus an abstract class for a skeletal implementation**.

7.10 Marker and Functional Interfaces

Marker interfaces have **no methods**. Implementing one tags a class so that the JVM or a library treats it specially.

Marker interface	Meaning
`java.io.Serializable`	Objects may be serialized (Phase 12)
`java.lang.Cloneable`	`Object.clone()` is allowed; otherwise `CloneNotSupportedException` (Phase 8)
`java.util.RandomAccess`	The List supports fast index access (`ArrayList` yes, `LinkedList` no)

A user-defined marker interface means nothing to the JVM itself, though your own code can check it with `instanceof`. Today, **annotations** (`@Entity`, `@Test`) are the usual way to attach metadata.

Functional interfaces have **exactly one abstract method** (default and static methods are allowed). They can be implemented with a **lambda** or a **method reference** (Java 8). The optional `@FunctionalInterface` annotation makes the compiler enforce the single-abstract-method rule.

```
interface InterestCalculator {
    double calculate(double principal);
}

InterestCalculator simple = p -> p * 0.05;                    // lambda
InterestCalculator anon = new InterestCalculator() {          // pre-Java 8 equivalent
    public double calculate(double p) { return p * 0.05; }
};
System.out.println(simple.calculate(10000));                  // 500.0
```

Common built-in functional interfaces include `Runnable`, `Callable`, `Comparator`, `Function<T,R>`, `Predicate<T>`, `Supplier<T>`, `Consumer<T>` and `BiFunction`. They are covered with streams in Phase 15.

7.11 Predict the Output

```
class A { void show() { System.out.println("A"); } }
class B extends A { void show() { System.out.println("B"); } }
A obj = new B();
obj.show();                                          // B

static void print(Object o) { System.out.println("Object"); }
static void print(String s) { System.out.println("String"); }
A ref = new B(); Object str = "x";
print(str);                                          // Object (overload chosen by declared type)
print(null);                                         // String

static void num(double d)  { System.out.println("double"); }
static void num(Integer i) { System.out.println("Integer"); }
num(10);                                             // double (widening before boxing)

interface Greeter { default String hi() { return "Hello"; } }
class Polite implements Greeter { public String hi() { return "Namaste"; } }
System.out.println(new Polite().hi());               // Namaste (class wins over default)

A a2 = new A();
System.out.println(a2 instanceof B);                 // false
B b2 = (B) a2;                                       // ClassCastException
```

7.12 Interview Questions

Q. What is polymorphism? What are its types?

The ability of one interface or method name to take many forms. Compile-time polymorphism is achieved by overloading and resolved by the compiler. Runtime polymorphism is achieved by overriding and resolved by the JVM using the actual object type.

Q. Can we overload by changing only the return type?

No. Calls would be ambiguous because the return type isn't part of the signature. The parameter lists must differ.

Q. Can static, private or final methods be overridden?

No. Static methods are hidden (chosen by reference type). Private methods aren't inherited. Final methods are explicitly locked. All three are statically bound.

Q. What are the rules for overriding regarding access and exceptions?

Access must be the same or wider. Checked exceptions must be the same, narrower or none; unchecked exceptions are unrestricted. The return type must be the same or covariant.

Q. What is a covariant return type?

An overriding method may return a subtype of the overridden method's return type (Java 5+), for example `Dog reproduce()` overriding `Animal reproduce()`.

Q. What is dynamic method dispatch?

The mechanism by which a call through a parent or interface reference executes the overriding method of the actual runtime object.

Q. What is the difference between upcasting and downcasting?

Upcasting (child → parent) is implicit and always safe. Downcasting (parent → child) needs an explicit cast and throws `ClassCastException` if the object isn't that type. Guard it with `instanceof` (pattern matching in Java 16+).

Q. Does `final` make an object immutable?

No. A final reference can't be reassigned, but the object it points to can still change. A final class can't be subclassed but can still be mutable.

Q. Can an abstract class have a constructor? Can it be instantiated?

Yes, it can have constructors; they run when subclasses are created. It can't be instantiated directly.

Q. What are the differences between an abstract class and an interface? When would you use each?

Abstract classes can have state, constructors and any access modifiers, and a class can extend only one. Interfaces define contracts with constants plus abstract/default/static/private methods, and a class can implement many. Use an abstract class for a shared base implementation in an IS-A hierarchy; use an interface for capabilities and for decoupling.

Q. Why were default methods introduced?

To let existing interfaces evolve (for example adding `stream()` to `Collection`) without breaking existing implementing classes. They also provide multiple inheritance of behaviour.

Q. How do you resolve two interfaces with the same default method?

The implementing class must override the method. It can delegate to one or both versions with `InterfaceName.super.method()`.

Q. What are marker and functional interfaces?

A marker interface has no methods and tags a class (`Serializable`, `Cloneable`). A functional interface has exactly one abstract method and can be implemented by a lambda (`Runnable`, `Comparator`, `Function`).

7.13 Quick Revision

- **Overloading**: same name, different parameters, compile time. Resolution: exact → widening → boxing → varargs, most specific wins; based on **declared** types.
- **Overriding**: same signature, runtime. Covariant return OK; access same or wider; no broader checked exceptions; always use `@Override`.
- Static methods are **hidden**; private, static and final methods and **fields** are statically bound.
- Upcast is implicit; downcast is explicit and risks `ClassCastException`. Use `instanceof` pattern matching.
- `final`: variable → assign once; method → no override; class → no subclass. **Final is not the same as immutable.**
- Abstract class: no `new`, may have constructors, state and concrete methods; subclasses implement the abstract methods or are abstract themselves.
- Interface: constants are `public static final`; methods are public abstract, plus `default`/`static` (8) and `private` (9). A class implements many interfaces.
- Default-method conflict → override and pick with `X.super.m()`. Class methods beat interface defaults.
- Program to interfaces for loose coupling and mockable tests.
- Marker interface = no methods; functional interface = one abstract method → lambdas.